UNIVERSIDAD AUTÓNOMA METROPOLITANA
DIVISIÓN DE CIENCIAS BÁSICAS E INGENIERÍA
DEPARTAMENTO DE FÍSICA

REDES NEURONALES PARA LA TRANSICIÓN DE FASE FUERTE II

PROYECTO TERMINAL II
INVESTIGACIÓN TEÓRICA

PARA OBTENER EL TÍTULO DE LICENCIADO EN FÍSICA

PRESENTA

EMMANUEL YAEL PÉREZ GONZÁLEZ
MATRÍCULA: 2203007749

LEONARDO YAEL PÉREZ JIMÉNEZ
MATRÍCULA: 2203042659

DR. LUIS ALBERTO HERNÁNDEZ ROSAS
ASESOR A CARGO

DR. MARCO ANTONIO MACEDA SANTAMARÍA
COORDINADOR DE LA LICENCIATURA

Ciudad de México, México
August 13, 2025

Este proyecto terminal esta dedicado a nuestra familia, amigos y parejas, además, de las personas que nos apoyaron y creyeron en nosotros, puesto que sin ustedes tanto este proyecto terminal como la licenciatura no se hubieran finalizado. Los queremos mucho.

Gracias.

Índice

1 Introducción **4**

2 Motivación **5**

3 Teoría de Yukawa **6**

4 Cálculo del potencial efectivo **8**
4.1 Correcciones de anillo 9
4.2 Correcciones a un lazo para la parte bosónica y fermiónica 12
4.3 Potencial efectivo 15
4.4 Estabilización del vacío 16

5 Diagrama de fase (Metodología) **17**

6 Resultados **20**
6.1 Gráficas 21

7 Conclusión **25**

A Apéndice **26**

Resumen

En el presente proyecto terminal se obtendrán las combinaciones de parámetros libres m, g y λ del modelo de Yukawa, donde λ que es la constante acoplamiento de auto-interacción de los campos escalares, m la masa del bosón escalar y g que es el parámetro que cuantifica la intensidad de la interacción entre el campo escalar; donde se busca reproducir la curvatura de la linea de transición de fase reportada por Lattice QCD [1], mediante técnicas de optimización de parámetros, en este caso *differential evolution*, en el lenguaje de Python y tener un conjunto amplio de combinaciones de estos parámetros que satisfacen dichas condiciones.

Palabras clave

Teoría cuántica de campos, QCD, transición de fase, Yukawa, Python, optimización de parámetros.

1. Introducción

El objetivo de este proyecto es explorar un problema contemporáneo de altas energías, el cual esta relacionado al punto crítico del diagrama de fase de la Cromodinámica Cuántica (QCD), asociado a la transición de fase entre la materia hadrónica y el plasma de quarks y gluones. Experimentos de colicionadores de hadrónes de iones pesados buscan completar el diagrama de fase de la QCD, simulaciones en Lattice QCD (LQCD) indican que la transición de confinamiento-desconfinamiento es un *crossover* con temperatura pseudocrítica de aproximadamente 158 MeV a potencial químico bariónico μ_B=0 (donde el potencial químico bariónico es 3 veces el potencial químico de quarks μ_q), lo que significa que no hay una temperatura crítica específica a la que aparece esa transición. Estudios recientes también interpretan este *crossover* como el remanente de una transición de segundo orden que permite mapear los parámetros universales y extraer la dependencia con el potencial químico bariónico [1].

El modelo de Yukawa, se usa para estudiar cómo la simetría quiral de la Cromodinámica Cuántica se restaura a altas temperaturas o densidades. Este modelo está construido para explicar y reproducir la misma curva que la QCD y se ha mostrado que produce una buena descripción de la transición fase de la QCD y de las propiedades de distribución de equilibrio, las cuales se desvían de las del modelo de gas de resonancia hadrónica. La desviación que acabamos de mencionar se debe a la inclusión de propiedades de apantallamiento en el plasma, las cuales están codificadas en las contribuciones de anillo del plasma [2].

Para estudiar la restauración de la simetría quiral a temperatura y densidad bariónicas finitas, calculamos el potencial efectivo como función del campo escalar ϕ. El potencial se compone del término de orden árbol, correcciones a un lazo y los diagramas de anillo. El término orden árbol proviene del Lagrangiano original, tiene la forma de sombrero mexicano y su mínimo determina el valor de ϕ en el vacío. Los términos de correcciones de lazo surgen mediante la cuantización de los quarks y los mesones, dependen de la temperatura T y del potencial químico de quarks μ_q. Estas correcciones incorporan los efectos térmicos. Para el caso de los diagramas de anillo, estos son un conjunto infinito de diagramas de lazo que son las soluciones estacionarias; es decir, de la energía libre cuando incorporas correcciones de interacción y que te permiten tener los efectos de un plasma [3].

Es importante destacar en la teoría de Yukawa, la restauración de la simetría quiral ya que, cuando se aumenta la temperatura, el mínimo del potencial efectivo se desplaza hacia $\phi = 0$, lo cual indica la restauración de la simetría. La inclusión de correcciones de un lazo y de diagramas de anillo permiten describir cómo la simetría quiral se rompe y se restaura en función de la temperatura o la densidad bariónica [2].

En este trabajo empezaremos dando un contexto general del modelo de Yukawa, correcciones de anillo, correcciones a un lazo, el potencial efectivo y la estabilización del vacío; después hablaremos de los parámetros, m, g y λ ,existentes que reproducen la curva del diagrama de fase de QCD; donde m es el valor del vacío del campo y suele fijarse de manera que

reproduzca el valor del condensado de plasma en el vacío; g es acoplamiento de Yukawa, que es es el parámetro que cuantifica la fuerza de interacción entre el campo escalar ϕ y el campo fermiónico ψ; y por ultimo el parámetro λ es el acoplamiento de auto-interacción de los campos escalares [4].

Para después dar nuestra motivación para encontrar una mayor combinación de estos parámetros y a continuación mencionaremos como logramos obtener estas combinaciones a través de técnicas especificas de Python, en específico el optimizado *differential evolution* con una función de error de error cuadrático medio, así como el porque los escogimos. Y después se presentarán los resultados obtenidos y nuestras respectivas conclusiones.

2. Motivación

La teoría de Yukawa es un modelo efectivo simple, que ayuda a describir la dinámica de la interacción fuerte a bajas energías, y que reproduce el rompimiento espontáneo de la simetría quiral de QCD, por lo tanto se puede emplear para estudiar la transición de fase de sistemas fuertemente interactuantes, permitiendo así explorar el diagrama de fase de QCD de forma efectiva. En esta teoría tenemos ciertos parámetros que no están determinados, estos son m, g y λ y están presentes en el Lagrangiano, del cual hablaremos en el siguiente capitulo. Existen ya ciertas combinaciones de estos parámetros, que reproducen esta curva, sin embargo, sabemos que no son los únicos y podrían existir muchos más y eso nos motiva en este trabajo a hacer una búsqueda exhaustiva de todas las posibilidades que pueden tener estos valores.

Estos valores correspondientes a nuestros parámetros se obtienen para reproducir la curva de transición de un resultado de LQCD: Esos parámetros se fijan con condiciones físicas y deben de satisfacer dichas condiciones del modelo, como las condiciones de ruptura espontánea de la simetría, la transición de fase entre la materia hadrónica y el plasma de quarks y gluones, entre otras [1].

Este proceso se puede hacer con diferentes técnicas y durante los últimos años se han obtenido muy pocas combinaciones de parámetros λ, m y g que reproduzcan satisfactoriamente la curva de Lattice QCD. El propósito de este proyecto terminal es, mediante técnicas de optimización numérica, encontrar la mayor cantidad de parámetros que satisfagan ciertas condiciones físicas: Para este trabajo vamos a pedir que la curva de transición que obtengamos siga la curvatura dada por LQCD y así dar las bases para la obtención de parámetros y un avance físico en la teoría. Vamos a partir del modelo de Yukawa, porque es un modelo más sencillo al tener solo un campo escalar y así asimilar mejor los conocimientos y después trasladarlo a modelos más complejos [2].

3. Teoría de Yukawa

La teoría de Yukawa, nombrada en honor al físico japonés Hideki Yukawa, inicialmente fue concebida para describir la fuerza fuerte que une a los nucleones (protones y neutrones) dentro del núcleo atómico. En este contexto, la fuerza nuclear es mediada por el intercambio de piones, que son mesones pseudoescalares [4].

El Lagrangiano de este modelo es el siguiente

$$\mathcal{L}(\psi,\phi) = \frac{1}{2}(\partial_\mu\phi)^2 + \frac{m^2}{2}\phi^2 - \frac{\lambda}{4}\phi^4 + i\bar{\psi}\gamma^\mu\partial_\mu\psi - g\bar{\psi}\phi\psi, \tag{3.1}$$

donde ϕ describe el campo escalar, los términos ψ y $\bar{\psi}$ son el espinor y el espinor adjunto de Dirac [4], estos describen el campo fermiónico, mientras que los parámetros que aparecen son, λ que es la constante acoplamiento de auto-interacción de los campos escalares, m es el parámetro de masa del bosón escalar y g que es el parámetro que cuantifica la fuerza de interacción entre el campo escalar ϕ y el campo fermiónico ψ. Ahora bien, el término $\frac{1}{2}(\partial_\mu\phi)^2 + \frac{m^2}{2}\phi^2$ representa la dinámica libre del campo escalar, donde el primer término es la parte cinética del bosón y el segundo término es el de masa [4], el término $\frac{\lambda}{4!}\phi$ describe la auto-interacción del campo escalar, el término $i\bar{\psi}\gamma^\mu\partial_\mu\psi$ es la energía cinética del fermión, mientras que γ^μ son la matrices de Dirac [4], por último, tenemos que $g\,\bar{\psi}\,\phi\,\psi$ es el término que describe el acoplamiento directo entre el campo fermiónico y el campo escalar (término de interacción de Yukawa).

La interacción de Yukawa es responsable de otorgar masa a los fermiones fundamentales, debido a la ruptura espontánea de simetría del potencial, con el fin de mostrar el rompimiento espontáneo de la simetría de este modelo tenemos que hacer que nuestro campo escalar ϕ tenga un valor de expectación en el vacío de v, para ello tenemos que hacer un desplazamiento

$$\phi \to \phi + v, \tag{3.2}$$

que, sustituyendo en el Lagrangiano de nuestro modelo, la Ec. (3.1) se escribe de la siguiente forma

$$
\begin{aligned}
\mathcal{L} =& \frac{1}{2}(\partial_\mu(\phi+v))^2 + \frac{m^2}{2}(\phi+v)^2 - \frac{\lambda}{4}(\phi+v)^4 + i\bar{\psi}\gamma^\mu\partial_\mu\psi - g\bar{\psi}(\phi+v)\psi \\
=& \frac{1}{2}(\partial_\mu\phi)^2 - \frac{(3\lambda v^2 - m^2)}{2}\phi^2 - \frac{\lambda}{4}(\phi^4 + 4\phi^3 v + 4\phi v^3) + \bar{\psi}(i\gamma^\mu\partial_\mu - gv)\psi \\
& - g\bar{\psi}\phi\psi + \frac{m^2}{2}v^2 - \frac{\lambda}{4}v^4 \\
=& \frac{1}{2}(\partial_\mu\phi)^2 - \frac{m_\phi^2}{2}\phi^2 - \frac{\lambda}{4}(\phi^4 + 4\phi^3 v + 4\phi v^3) + \bar{\psi}(i\gamma^\mu\partial_\mu - m_f)\psi \\
& - g\bar{\psi}\phi\psi + \frac{m^2}{2}v^2 - \frac{\lambda}{4}v^4,
\end{aligned}
\tag{3.3}
$$

donde $m_\phi^2 = 3\lambda v^2 - m^2$ y $m_f = gv$ son las masas dinámicas del bosón y el fermión, respectivamente. De la ecuación anterior podemos identificar que

$$
V_{tree} = -\frac{m^2}{2}\phi^2 + \frac{\lambda}{4}\phi^4, \tag{3.4}
$$

es el potencial a orden árbol, este término es la clave que permite entender el rompimiento de la simetría, ya que este potencial genera un mínimo llamado valor de expectación en el vacío del campo escalar ϕ, cuyo valor se puede calcular derivando V_{tree} respecto de ϕ e igualando a 0, como se muestra a continuación

$$
0 = -m^2\phi + \lambda\phi^3, \tag{3.5}
$$

como podemos observar tenemos 3 soluciones, $\phi = 0$, $\phi = -\sqrt{\frac{m^2}{\lambda}}$ y $\phi = \sqrt{\frac{m^2}{\lambda}}$, esta ultima solución es la que nos interesa, ya que nos regresa un valor positivo evaluando en la segunda derivada, lo que quiere decir que es el mínimo de nuestra función, entonces la definimos como

$$
v_0 = \sqrt{\frac{m^2}{\lambda}}. \tag{3.6}
$$

Ahora bien, si sustituimos este resultado en nuestras masas dinámicas tenemos que

$$
\begin{aligned}
m_\phi^2(v_o) &= 3\lambda(\frac{m^2}{\lambda}) - m^2 = 2m^2, \\
m_f(v_0) &= g\sqrt{\frac{m^2}{\lambda}},
\end{aligned}
\tag{3.7}
$$

de estos resultados podemos notar que, al romper espontáneamente la simetría y movernos al mínimo de nuestro potencial mediante la Ec. (3.2), el bosón y el fermión adquirieron masa en nuestra teoría y tales valores dependen de los parámetros que deseamos encontrar, de este modo podemos entender aún más la relevancia de estos parámetros.

4. Cálculo del potencial efectivo

Para el objetivo del proyecto es importante calcular las contribuciones a primer lazo de nuestra energía potencial de la parte bosónica V_b^1 y la parte fermiónica V_f^1, recordando que estas contribuciones se calculan a partir de una expansión en series de potencias de la acción $S(\phi, \psi)$ y se buscan sus soluciones estacionarias orden a orden, donde a primer orden esta es la solución [4],

$$V_b^1 = -\frac{i}{2}\Omega Tr[\ln G^{-1}], \tag{4.1}$$

donde Ω es el volumen espacio-temporal total, la traza indica suma/integral sobre todos los grados de libertad de nuestra teoría como veremos mas adelante y G es el propagador o bien es la función de green a dos puntos y esta definida como, que para este caso tendrá incluida la masa dinámica m_ϕ^2, el cual esta definido en el espacio de momentos como

$$\langle k|G^{-1}|k^1\rangle = G^{-1}(k, k^{-1}) = (-k^2 + m_\phi^2)\delta(k - k'), \tag{4.2}$$

y que la traza se puede escribir como la integral en el espacio de configuraciones como

$$\begin{aligned} \mathrm{Tr}\ln G^{-1} &= \int d^4x\, d^4k\, d^4k'\, \langle x|k\rangle\langle k|\ln G^{-1}|k'\rangle\langle k'|x\rangle \\ &= \int \frac{d^4x\, d^4k\, d^4k'}{(2\pi)^4} e^{-i(k-k')\cdot x} \ln\left(-k^2 + m_\phi^2\right) \delta^4(k - k') \\ &= \Omega \int \frac{d^4k}{(2\pi)^4} \ln\left(-k^2 + m_\phi^2\right), \end{aligned} \tag{4.3}$$

con este resultado podemos escribir la primera corrección al potencial que corresponde a la primera contribución bosonica dada la Ec.(4.1) como

$$V_b^1 = -\frac{i}{2}\int \frac{d^4k}{(2\pi)^4} \ln\left(-k^2 + m_\phi^2\right), \tag{4.4}$$

ahora bien, si pasamos al formalismo de Matsubara se tiene que

$$\int \frac{d^4k}{(2\pi)^4} \rightarrow \quad iT\sum_n \int \frac{d^3k}{(2\pi)^3}, \tag{4.5}$$

en este formalismo la componente cero k_0 de cuadrivector k adquiere el valor $i\omega_n$ que son las frecuencias de Matsubara para bosones y $i\tilde{\omega}_n$ para fermiones, con los siguientes valores

$$\omega_n = 2\pi n T \quad \text{y} \quad \tilde{\omega}_n = (2n+1)\pi T, \tag{4.6}$$

entonces podemos escribir la Ec. (4.4) como

$$V_b^1 = \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln\left(\omega_n^2 + \vec{k}^2 + m_\phi^2\right) = \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln G^{-1}. \tag{4.7}$$

De manera análoga al cálculo anterior, la corrección a un lazo de la contribución fermiónica de espín 1/2 en el formalismo de Matsubara sería

$$V_f^1 = -T\sum_n \int \frac{d^3k}{(2\pi)^3} Tr[\ln S^{-1}], \tag{4.8}$$

donde S el propagador del campo fermiónico también esta formalismo de Matsubara y con potencial químico quark finito, tal que

$$S(\tilde{\omega}_n, \mu_q, \vec{k}) = \frac{1}{\gamma_0(\tilde{\omega}_n - i\mu_q) + \not{k} + m_f}, \tag{4.9}$$

donde μ_q es el potencial químico de quark, que aparece debido a que estamos trabajando con un modelo con densidad de partículas finitas, se introduce para controlar el número medio de partículas del sistema, actuando como un parámetro termodinámico [5] y $\not{k}$ es el producto entre las componentes del cuadrivector k y las componentes de las matrices de Dirac, como se muestra a continuación

$$\not{k} = \sum_{i=1}^{3} \gamma^i k_i. \tag{4.10}$$

4.1. Correcciones de anillo

En este modelo de muchas partículas surge la posibilidad de que el campo interactúe con otros campos presentes en el medio. Estas interacciones generan correcciones cuánticas adicionales al propagador y se representan a través de los llamados diagramas de anillo ver figura (1);

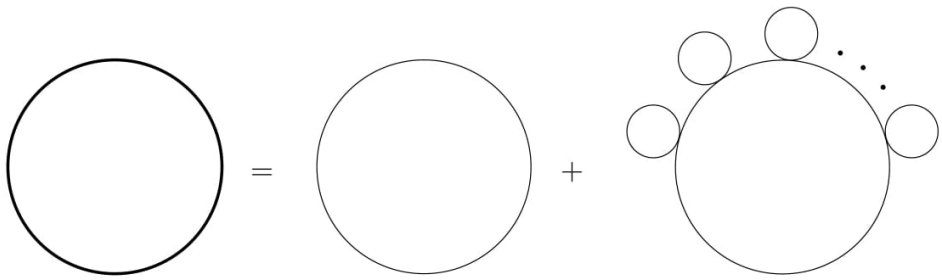

Figura 1: Diagramas de anillo [3]

donde cada lazo indica una interacción que esta asociadas con la autoenergía y se resuman sobre todas las posibilidades, que son una infinidad, es importante agregar estas correcciones ya que es necesario considerar todas estas interacciones entre partículas en el medio [3] y están descritas de la siguiente manera en el formalismo de Matsubara

$$V_{ring} = \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln\left(1 + \Pi G\right), \tag{4.11}$$

donde Π es la auto-energía del campo, que es una corrección cuántica de la propagación de los campos debido a interacciones con otros campos o consigo mismo [3]; de manera general para nuestro modelo la auto-energía total es de la siguiente forma

$$\Pi = \Pi_b + \Pi_f, \tag{4.12}$$

donde Π_b es la auto-energía de la parte bosónica y se calcula de la siguiente manera

$$\Pi_b = \lambda \int \frac{d^4k}{(2\pi)^4} \frac{i}{k^2 - m_\phi^2 + i\epsilon}, \tag{4.13}$$

Pasando al formalismo de Matsubara se tiene que

$$\Pi_b = -\lambda T \sum_n \int \frac{d^3k}{(2\pi)^3} \frac{1}{(\omega_n^2 + \vec{k}^2 + m_\phi^2)}, \tag{4.14}$$

sumando sobre todas las frecuencias de Matsubara tenemos que

$$\Pi_b = -\lambda \int \frac{d^3k}{(2\pi)^3} \frac{1}{2\sqrt{\vec{k}^2 + m_\phi^2}} \left(1 + 2 \frac{1}{\exp\left(\frac{\sqrt{\vec{k}^2 + m_\phi^2}}{T}\right) - 1} \right), \tag{4.15}$$

la contribución del vacío tras ser renormalizada y tomando valores fijos permanece constante, por lo tanto no contribuye a la auto-energía; tomando en cuenta solo la parte térmica y haciendo regularización dimensional, donde sabemos que $d^dk = k^{d-1}dkd\Omega^d$ y $\int d\Omega^d = 2\pi^{d/2}/\Gamma(d/2)$ resulta en

$$\begin{aligned}\Pi_b =& -\lambda\int_{\infty}^{\infty}\frac{d^3k}{(2\pi)^3}\frac{1}{\sqrt{k^2+m_\phi^2}}\left(\frac{1}{\exp\left(\frac{\sqrt{k^2+m_\phi^2}}{T}\right)-1}\right)\\ =& -\lambda\int_{0}^{\infty}\frac{k^2dk}{2\pi^2}\frac{1}{\sqrt{k^2+m_\phi^2}}\left(\frac{1}{\exp\left(\frac{\sqrt{k^2+m_\phi^2}}{T}\right)-1}\right),\end{aligned} \tag{4.16}$$

tomando el límite cuando $T^2 >> m_\phi^2$

$$\begin{aligned}\Pi_b =& -\frac{\lambda}{2\pi^2}\int_0^{\infty}\frac{k^2dk}{k}\left(\frac{1}{\exp\left(\frac{k}{T}\right)-1}\right)\\ =& -\frac{\lambda}{2\pi^2}\int_0^{\infty}dk\,k\left(\frac{1}{\exp\left(\frac{k}{T}\right)-1}\right),\end{aligned} \tag{4.17}$$

haciendo el siguiente cambio de variable $x = k/T$

$$\Pi_b = \frac{\lambda}{2\pi^2}\int_0^{\infty}T^2dx\left(\frac{x}{\exp(x)-1}\right), \tag{4.18}$$

a partir del resultado anterior podemos usar la definición integral de la función zeta de Riemann $\zeta(z)$

$$\zeta(z) = \frac{1}{\Gamma(z)}\int_0^{\infty}\frac{u^{z-1}}{e^u-1}du, \tag{4.19}$$

con $z-1=1$ tememos que la Ec. (4.18) es

$$\Pi_b = \frac{\lambda T^2}{2\pi^2}\zeta(2)\Gamma(2), \tag{4.20}$$

sabiendo que $\Gamma(2)=1$ y $\zeta(2)=\frac{\pi^2}{6}$ tenemos que

$$\Pi_b = \frac{\lambda T^2}{2\pi^2}(\frac{\pi^2}{6})(1) = \frac{\lambda T^2}{12}. \tag{4.21}$$

Ahora bien, para el caso de la auto-energía fermiónica se calcula de manera análoga a la bosónica, recordando que hay que remplazar un signo global, las distribuciones de Bose-Einstein por las de Fermi-Dirac y tomar en cuenta la contribución de un potencia químico de quarks, haciendo eso en la aproximación a alta temperatura $T^2 >> m_f^2$ se llega a

$$\Pi_j(T,\mu_q) = -\frac{g^2T^2}{\pi^2}\left[\mathrm{Li}_2\Big(-e^{\mu_q/T}\Big) + \mathrm{Li}_2\Big(-e^{-\mu_q/T}\Big)\right], \tag{4.22}$$

donde Li_2 se refiere al polilogaritmo de orden 2, de este modo tenemos que nuestra auto-energía total Ec.(4.12) es de la forma

$$\Pi_{\text{total}} = \frac{\lambda T^2}{12} - \frac{g^2T^2}{\pi^2}\left[\mathrm{Li}_2\Big(-e^{\mu_q/T}\Big) + \mathrm{Li}_2\Big(-e^{-\mu_q/T}\Big)\right]. \tag{4.23}$$

Regresando con la Ec. (4.11), de las correcciones de anillo podemos hacer el siguiente análisis

$$\begin{aligned}
V_{ring} =& \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln\left(\frac{G^{-1}}{G^{-1}}(1+\Pi G)\right) \\
=& \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln\left(\frac{1}{G^{-1}}(G^{-1}+\Pi)\right) \\
=& \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} [\ln(G^{-1}+\Pi) - \ln G^{-1}] \\
=& \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln(G^{-1}+\Pi) - \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln G^{-1},
\end{aligned} \tag{4.24}$$

Es importante observar que el último término de la ecuación anterior es Ec.(3.7), que es la corrección a un lazo del bosón V_b^1, entonces tenemos que

$$V_{ring} = \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln(G^{-1}+\Pi) - V_b^1. \tag{4.25}$$

4.2. Correcciones a un lazo para la parte bosónica y fermiónica

Retomando la corrección a un lazo de la parte bosónica Ec.(4.7), podemos derivar y después integrar respecto al cuadrado de la masa m_ϕ^2

$$V_b^1 = \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \ln(\omega_n^2 + \vec{k}^2 + m_\phi^2) = \frac{T}{2}\sum_n \int \frac{d^3k}{(2\pi)^3} \int \frac{dm_\phi^2}{\omega_n^2 + \vec{k}^2 + m_\phi^2}, \tag{4.26}$$

sumando sobre las frecuencias de Matsubara tenemos que

$$V_b^1 = \frac{1}{2}\int \frac{d^3k}{(2\pi)^3}\int dm_\phi^2 \frac{1}{2\sqrt{\vec{k}^2+m_\phi^2}}\left(1+\frac{2}{e^{\frac{\sqrt{\vec{k}^2+m_\phi^2}}{T}}-1}\right), \tag{4.27}$$

en esta ecuación podemos observar dos términos, el primer término es el 1, que esta dentro del paréntesis, este esta multiplicando a un término que no depende de la temperatura, a esto le llamamos la contribución de vacío, mientras que el segundo término si tiene dependencia térmica.

Para empezar vamos a trabajar con el término de vacío, primero vamos a realizar regularización dimensional, haciendo $3 \to d$, entonces se tiene lo siguiente

$$V_{b,0}^1 = \frac{1}{2}\int \frac{d^3k}{(2\pi)^3}\int dm_\phi^2 \frac{1}{2\sqrt{\vec{k}^2+m_\phi^2}} \to \frac{1}{2}\int \frac{d^dk}{(2\pi)^d}\int dm_\phi^2 \frac{1}{2\sqrt{\vec{k}^2+m_\phi^2}}, \tag{4.28}$$

para mantener la dimensión añadimos una constate μ

$$V_{b,0}^1 = \frac{\mu^{3-d}}{4}\int \frac{d^dk}{(2\pi)^d}\int dm_\phi^2 \frac{1}{\sqrt{\vec{k}^2+m_\phi^2}}, \tag{4.29}$$

ahora integramos respecto a m_ϕ^2

$$V_{b,0}^1 = \frac{\mu^{3-d}}{2}\int \frac{d^dk}{(2\pi)^d}\sqrt{\vec{k}^2+m_\phi^2} = \frac{\mu^{3-d}}{2}\int \frac{d^dk}{(2\pi)^d}\frac{1}{(\vec{k}^2+m_\phi^2)^{-1/2}}, \tag{4.30}$$

existe una forma general para resolver este tipo de integrales [4], la cual es de la siguiente manera

$$\int \frac{d^d\ell}{(2\pi)^d(\ell+\Delta)^n} = \frac{1}{(4\pi)^{d/2}}\frac{\Gamma(n-d/2)}{\Gamma(n)}\left(\frac{1}{\Delta}\right)^{n-d/2}, \tag{4.31}$$

si tomamos $n=-1/2$ y $\Delta = m_\phi^2$, tenemos que

$$V_{b,0}^1 = \frac{\mu^{3-d}}{2}\frac{1}{(4\pi)^{d/2}}\frac{\Gamma\left(-\frac{1+d}{2}\right)}{\Gamma\left(-\frac{1}{2}\right)}\left(\frac{1}{m_\phi^2}\right)^{-\frac{1+d}{2}}, \tag{4.32}$$

ahora hay que aproximar nuestra dimensión a 3 de la siguiente manera $d \to 3-2\epsilon$, entonces

$$V_{b,0}^1 = \frac{\mu^{2\epsilon}}{2}\frac{1}{(4\pi)^{\frac{3}{2}-\epsilon}}\frac{\Gamma(-2+\epsilon)}{\Gamma(-1/2)}\left(\frac{1}{m_\phi^2}\right)^{-2+\epsilon}, \tag{4.33}$$

tomando el límite $\lim \epsilon \to 0$

$$V_{b,0}^1 = \frac{m_\phi^4}{128\pi^2}\left[-\frac{2}{\epsilon} + (-3+2\gamma_E) - 2\ln\left(\frac{4\pi\mu^2}{m_\phi^2}\right)\right], \tag{4.34}$$

aplicamos el esquema de sustracción mínima modificado $\overline{MS}$, para absorber las divergencias y también las constantes γ_E y $\ln(4\pi)$, de tal manera que tenemos nuestra corrección a un lazo de la parte bosónica en el vacío

$$V_{b,0}^1 = -\frac{m_\phi^4}{64\pi^2}\left[\frac{3}{2} + \ln\left(\frac{\mu^2}{m_\phi^2}\right)\right], \tag{4.35}$$

donde μ es la constante de renormalización que nos dice que nuestra teoría se volvió una teoría efectiva. Ya que nos dice que hasta este valor energético tiene validez nuestra teoría.

Continuando con la parte térmica de la Ec. (4.27)

$$V_{b,T}^1 = \frac{1}{2}\int \frac{d^3k}{(2\pi)^3}\int dm_\phi^2 \frac{1}{\sqrt{\vec{k}^2+m_\phi^2}}\left(\frac{1}{e^{\frac{\sqrt{\vec{k}^2+m_\phi^2}}{T}}-1}\right), \tag{4.36}$$

si integramos respecto de m_ϕ^2, haciendo un cambio de variable donde $x = (\sqrt{\vec{k}^2+m_\phi^2})/T$ y $dx = -\frac{dm_\phi^2}{2T(\sqrt{\vec{k}^2+m_\phi^2})}$

$$V_{b,T}^1 = T\int \frac{d^3k}{(2\pi)^3}\ln\left(1-e^{-\sqrt{k^2+m_\phi^2}/T}\right), \tag{4.37}$$

ahora podemos hacer regularización dimensional usando $d^dk = k^{d-1}dkd\Omega^d$ y $\int d\Omega^d = 2\pi^{d/2}/\Gamma(d/2)$,

$$V_{b,T}^1 = \frac{T}{2\pi^2}\int_0^\infty dk\ k^2 \ln\left(1-e^{-\sqrt{k^2+m_\phi^2}/T}\right), \tag{4.38}$$

siendo este la contribución térmica de la corrección a un lazo de la parte bosónica. Sumando la contribución térmica Ec.(4.38) y la del vacío Ec.(4.35) tenemos que

$$V_b^1 = -\frac{m_\phi^4}{64\pi^2}\left[\frac{3}{2} + \ln\left(\frac{\mu^2}{m_\phi^2}\right)\right] + \frac{T}{2\pi^2}\int_0^\infty dk\ k^2 \ln\left[1-e^{-\sqrt{k^2+m_\phi^2}/T}\right]. \tag{4.39}$$

Para el caso de la parte fermiónica es un proceso análogo al que hicimos con la parte bosónica, cuyo resultado para el vacío es [1]:

$$V^1_{f,0} = \frac{m_f^4}{16\pi^2}\left[\ln\left(\frac{\mu^2}{m_f^2}\right) + \frac{3}{2}\right], \tag{4.40}$$

mientras que para la parte térmica se tiene que [1]:

$$\begin{aligned} V^1_{f,T} = &-\frac{T}{\pi^2}\int_0^\infty d_k k^2 \Bigg(\ln\left[1 + e^{-\frac{\sqrt{k^2+m_f^2}-\mu_q}{T}}\right] \\ &+ \ln\left[1 + e^{-\frac{\sqrt{k^2+m_f^2}+\mu_q}{T}}\right]\Bigg), \end{aligned} \tag{4.41}$$

de tal forma que la corrección cuántica de la parte fermiónica es

$$\begin{aligned} V^1_f =& \frac{m_f^4}{16\pi^2}\left[\ln\left(\frac{\mu^2}{m_f^2}\right) + \frac{3}{2}\right] \\ &- \frac{T}{\pi^2}\int_0^\infty d_k k^2 \Bigg(\ln\left[1 + e^{-\frac{\sqrt{k^2+m_f^2}-\mu_q}{T}}\right] \\ &+ \ln\left[1 + e^{-\frac{\sqrt{k^2+m_f^2}+\mu_q}{T}}\right]\Bigg), \end{aligned} \tag{4.42}$$

4.3. Potencial efectivo

Una vez calculadas todas las correcciones cuánticas de nuestro potencial podemos sumarlas y escribir el potencial efectivo que vamos a usar, recordando que para el potencial a orden árbol, un lazo bosónico, un lazo fermiónico y los anillos sus expresiones corresponden a las ecuaciones (3.4), (4.39), (4.42) y (4.25), respectivamente, sumando todo tenemos que el potencia efectivo se escribe como

$$V_{eff} = V_{\text{tree}} + V_b + V_f + V_{\text{ring}}, \tag{4.43}$$

de tal forma que nos queda la siguiente expresión

$$
\begin{aligned}
V_{\text{eff}} = & -\frac{m^2}{2}\,\phi^2 + \frac{\lambda}{4}\,\phi^4 \\
& - \frac{m_\phi^4}{64\pi^2}\left[\frac{3}{2} + \ln\left(\frac{\mu^2}{m_\phi^2}\right)\right] \; + \; \frac{T}{2\pi^2}\int_0^\infty dk\, k^2 \; \ln\left[1 - e^{-\frac{\sqrt{k^2+m_\phi^2}+\Pi}{T}}\right] \\
& + \frac{m_f^4}{16\pi^2}\left[\ln\left(\frac{\mu^2}{m_f^2}\right) + \frac{3}{2}\right] \; - \; \frac{T}{\pi^2}\int_0^\infty dk\, k^2 \left\{ \ln\left[1 + e^{-\frac{\sqrt{k^2+m_f^2}-\mu_q}{T}}\right]\right. \\
& \left. + \; \ln\left[1 + e^{-\frac{\sqrt{k^2+m_f^2}+\mu_q}{T}}\right]\right\}.
\end{aligned}
\tag{4.44}
$$

se debe señalar que el segundo término de las correcciones de anillo se anula con la corrección bosonica a un lazo, pero dado que el término sobrante de las correcciones de anillo es similar a la misma de la corrección bosónica, con la diferencia de que $m_\phi^2 \to m_\phi^2 + \Pi$, tenemos que son resultados análogos con la diferencia de que la parte térmica de esta corrección aparece en el término de auto-energía.

4.4. Estabilización del vacío

Es importante aclarar que, en el vacío donde no hay contribuciones térmicas, queremos que nuestro potencial efectivo no modifique el mínimo a orden arbol de la teoría, pero debido a que existen correcciones cuánticas en el vacío es necesario agregar términos que lo estabilicen, además de que nos permitan no depender de la escala de renormalización del vacío que obtuvimos al tratar con las correcciones de vacío, para el caso del campo bosónico así como para el fermiónico, serían δm^2 y $\delta\lambda^2$, de tal forma que de la ecuación (4.44) tenemos que

$$
V_{\text{vac}} = -\frac{m^2 + \delta m^2}{2}\phi^2 + \frac{\lambda + \delta\lambda^2}{4}\phi^4 - \frac{m_\phi^4}{64\pi^2}\left(\frac{3}{2} + \ln\left(\frac{\mu^2}{m_\phi^2}\right)\right) + \frac{m_f^4}{16\pi^2}\left(\ln\left(\frac{\mu^2}{m_f^2}\right) + \frac{3}{2}\right),
\tag{4.45}
$$

para lograr mantener el valor del mínimo de la contribución sin correcciones cuánticas (potencial orden árbol), se deben cumplir las mismas condiciones que tiene este, las cuales son

$$
\left.\frac{\partial V_{\text{vac}}}{\partial \phi}\right|_{\phi=v_0} = 0,
\tag{4.46}
$$

y también que

$$
\left.\frac{\partial^2 V_{\text{vac}}}{\partial \phi^2}\right|_{\phi=v_0} = 2m^2,
\tag{4.47}
$$

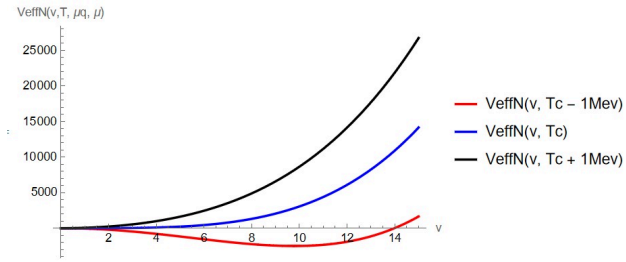

Figura 2: Representación esquemática de la ruptura y restauración de la simetría

una vez resueltas estas dos ecuaciones tenemos que los términos que estabilizan el vacío son los siguientes,

$$\begin{aligned} \delta m^2 =& \frac{m^2(4g^4 - 6\lambda^2 + 3\lambda^2 \ln\left(\frac{\mu}{2m^2}\right))}{16\pi^2\lambda}, \\ \delta\lambda^2 =& -\frac{-9\lambda^2 \ln\left(\frac{\mu^2}{2m^2}\right) + 4g^4 \ln\left(\frac{\lambda\mu}{m^2 g^2}\right)}{16\pi^2}. \end{aligned} \tag{4.48}$$

Como último comentario es importante mencionar que en la practica sustrajimos todas las constantes del potencial efectivo, esto se logró restando al mismo potencial el potencial cuando $\phi = 0$.

5. Diagrama de fase (Metodología)

Para poder encontrar la temperatura de transición tuvimos que calcular la segunda derivada de nuestro potencial efectivo definido anteriormente en la Eq.(4.44), ya que específicamente nos interesa saber cuando la curvatura de nuestro potencial efectivo en el origen es igual a cero, debido a que nos indica cuando la simetría se restaura, pues antes de ser restaurada nuestro potencial efectivo tiene una curvatura negativa; entonces igualamos a cero para poder encontrar la temperatura donde la curvatura es nula, que es la temperatura donde ocurre el rompimiento espontáneo de la simetría e indica una transición de fase, y se le conoce como temperatura critica.

Lo que queremos hacer es lo siguiente, queremos encontrar la curva de transición para distintos valores del potencial químico de quarks μ_q, entonces el método es el siguiente. Fijo el potencial químico, después ocupamos el criterio de la segunda derivada mencionada anteriormente, evaluada en cero para encontrar la curva de transición. La comprobamos haciendo el *plot* y vimos que en efecto es la temperatura de transición.

Junto con esta información sabemos como se debería de ver como función de μ_B, el cual es 3 veces el potencial químico μ_q esa temperatura de transición gracias a los resultados de Lattice QCD. La ecuación que debe de seguir es la siguiente [2]

$$\frac{T_c(\mu_B)}{T_c^0} = 1 - k_2 \left(\frac{\mu_B}{T_c^0}\right)^2 + k_4 \left(\frac{\mu_B}{T_c^0}\right)^4, \tag{5.1}$$

donde $k_2 = 0.0153$ y $k_4 = 0.00032$ son los valores los parámetros de la curvatura, μ_B el potencial bariónico y T_c^0 la temperatura de transición de fase a potencial bariónico cero.

Entonces, como nuestro objetivo es seguir la curva, para poder seguirla tenemos que restringir en los parámetros libres que tenemos m, g y λ. Ahí es cuando imponemos estas condiciones en nuestro programa de Python para minimizar el error y buscar todos estos conjuntos de parámetros que nos ayuden a seguir esta curva.

Para eso discretizamos la curva de la transición de fase, en nuestros valores específicos escogidos de potencial químico bariónico μ_B, para después, a través de un programa en Python, comparar nuestras temperaturas de transición de nuestro modelo con reportadas en el artículo de Lattice QCD [2]. Y mediante una función de pérdida y optimización se minimice este error y estas temperaturas de transición de fase sean lo más parecidas posibles entre ellas.

Debido a que nuestros potenciales no tienen solución analítica es indispensable usar métodos numéricos, como en nuestro caso Gauss-Legendre, que es un método de integración numérica que ofrece varias ventajas significativas como eficiencia y precisión en problemas más complicados sobre otros métodos numéricos tradicionales como Simpson o trapecio. Estas ventajas son debido a que una cuadratura de Gauss-Legendre de n puntos puede calcular el valor casi exacto de la integral para cualquier polinomio de grado hasta 2n-1, en contraste con otros métodos numéricos. Por ejemplo, Simpson solo calcula con buena precisión hasta grado 3 y trapecio hasta grado 1. Es así como Gauss-Legendre nos proporciona una buena precisión y eficiencia computacional.

La regla de Gauus-Legendre o cuadratura de Gauss-Legendre toma la siguiente forma

$$\int_{-1}^{1} f(x)dx \sim \sum_{i=1}^{n} w_i f(x_i), \tag{5.2}$$

donde n es el número de puntos usados, w_i son los pesos y x_i son las raíces del n polinomio de Legendre. Y w_i está definido como

$$w_i = \frac{2}{(1 - x_i^2)[P_n'(x_i)]^2}, \tag{5.3}$$

donde $P_n(x)$ son los polinomios de Legendre de grado n y x_i son los nodos de los polinomios que son la i-ésima raiz de P_n. Por lo que $P_n'(x)$ es la derivada de el polinomio de Legendre evaluada en cada nodo. Hay que aclarar que para integrar nuestra función hay que fijar un intervalo de [-1,1] [6].

Para poder aproximar los datos experimentales de Lattice QCD y los de nuestro modelo, con el fin de encontrar parámetros óptimos que satisfagan la curva teórica usamos una función de pérdida (Error cuadrático medio) y un optimizador (*Differential evlolution*) de los cuales vamos a hablar a continuación..

En optimización o en aprendizaje automático, una función de pérdida o función de error es una medida numérica que te indica que tan mal está funcionando un modelo. Esta función tiene como entrada predicciones teóricas del modelo y valores experimentales o reales. Esta función de salida te regresa un número que mide el error y el objetivo de esta es minimizarla, lo que significa que, entre más disminuye el valor, la función está trabajando bien.

Para minimizar este error en el presente trabajo se usó el "error cuadrático medio" (MSE), que es un tipo de función de pérdida de la forma

$$MSE = \frac{1}{n}\sum_{i=1}^{2}(y_i - x_i)^2, \tag{5.4}$$

donde n es el número de datos, y_i es el valor real del dato y x_i es el valor predicho por el modelo o código.

Este modelo fue usado debido a que castiga más a los errores más grandes que a los pequeños (debido a la potencia cuadrada), es derivable (lo que facilita su uso y compatibilidad con otros métodos) y sobre todo es muy fácil de interpretar [7].

Explicación de *differential evolution* o evolución diferencial.

Differential evolution es un método metaheurístico de optimización estocástico, evolutivo y que no necesita derivar usado normalmente para la optimización global en espacios continuos de funciones complicadas (no lineales, multi-modales, con ruido, con muchos mínimos locales) que ha ganado reconocimiento por su sencillez, eficiencia y precisión en la resolución de estas.

Un método estocástico es una técnica que nos permite modelar sistemas que presentan aleatoriedad o mucho ruido de manera eficiente, debido a que se basa en el uso de variables aleatorias para representar magnitudes que varían en el tiempo o están en función de otra variable (lo cual está presente en nuestro problema).

Este método de optimización consta de cuatro fases:

Primero, la inicialización que consta de generar una población inicial de posibles soluciones de manera aleatoria, las cuales tienen una estructura de vectores. Segundo, la mutación; esto crea un nuevo vector mutante para cada solución actual, perturbándola con la diferencia ponderada de otros dos vectores de la población. Para poder entender mejor esto, imaginemos el caso más sencillo, tenemos tres puntos distintos que se escogen al azar: A , B y C. Calculamos la diferencia entre dos puntos, por ejemplo, $A - B$, después tomamos el otro punto restante C y damos un paso (F) en la dirección de esa diferencia, eso es a lo que llamaremos nuestro vector mutante. Después sigue la parte de la recombinación, donde se

tratan de mezclar las componentes de este vector mutante con las componentes del vector original anterior de manera aleatoria y generar un vector de prueba. Por último, la selección que consta de comparar el vector de prueba con el vector original y el que tenga la menor pérdida (o sea, el más apto) sobreviva para la siguiente interacción.

Las ventajas de este método es que permite encontrar parámetros óptimos globales de manera eficiente en problemas complicados o multidimensionales, la capacidad para resolver problemas no diferenciables, su robustez ante el ruido y que es paralelizable computacionalmente. Sin embargo, también tiene ciertas limitaciones como son el costo de las evaluaciones; si tienes funciones muy pesadas o complejas, como es el caso de este proyecto, los tiempos de ejecución pueden ser muy largos y es sensible a la escala de variables (conviene normalizar y acotar bien) [8].

6. Resultados

Los siguientes gráficos en 3D muestran que los parámetros se distribuyen en regiones en concretas por cada parámetro. Desde el primer valor de $\lambda = 0.6$, donde la región esta más concentrada y los parámetros están muy cerca entre sí, hasta los valores intermedios de λ donde suelen estar más separados. Los resultados obtenidos están de acuerdo con los de Lattice QCD, están dentro de los márgenes de error y básicamente todos las combinaciones de parámetros obtenidos fueron satisfactorias respecto a combinación la curva, con tan solo dos errores numéricos de novecientos datos, presentes para bajos valores de $\lambda = 0.6$ y $\lambda = 0.8$, que fueron sustraídos de los resultados. Esos errores numéricos no era que no satisficieran la curva, sino que presentaban un margen de error mayor, respecto a los de Lattice QCD.

Para ciertos parámetros de λ se puede observar como los parámetros m vs g se asemejan a una pequeña recta y se podría pensar que se podría tomar algún punto intermedio y satisfacer la curva, pero esto no es seguro. Se tendría que aumentar la estadística entre esas regiones específicas para una mejor conclusión.

Para estos resultados nosotros consideramos una región de búsqueda de parámetros m y g de $m = [50,150]$ y $g = [0.1,5]$, con el fin de obtener resultados solo dentro de esas regiones, ya que dentro de estas regiones existen algunas soluciones que se han encontrado de la curva de LQCD.

Las gráficas se obtuvieron aproximadamente en un tiempo de 66 horas con un total de novecientas combinaciones de parámetros obtenidos, lo cual fue de cien combinaciones por cada valor de λ desde 0.6 a 1.0 en saltos de 0.05.

6.1. Gráficas

Con lo que explicamos en el capitulo anterior ahora pasamos a mostrarles el resultado de nuestro análisis que hicimos, donde fijamos valores de λ desde 0.6 a 1 en pasos de 0.05, donde nos da cien combinaciones de parámetros por cada λ. Comenzaremos con dos gráficos en 3D a continuación. Con los parametros λ, m y g en los ejes x, y y z, respectivamente. Después mostraremos gráficos de m vs g para cada valor de λ para una visualización mejor de los datos por λ.

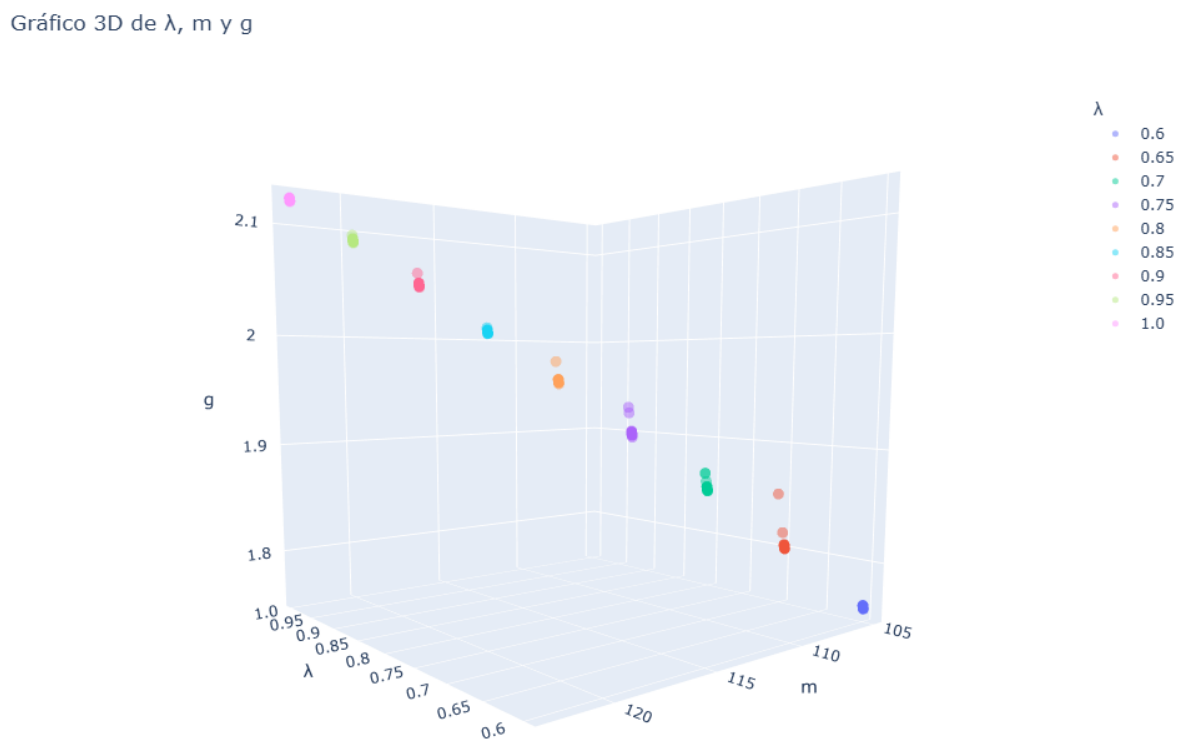

Figura 3: Gráfico 3D donde están todos las combinaciones de parámetros obtenidos desde λ igual a 0.6 hasta 1, en pasos de 0.05 con 100 combinaciones por λ. Donde en el eje vertical está g y en los horizontales λ vs m

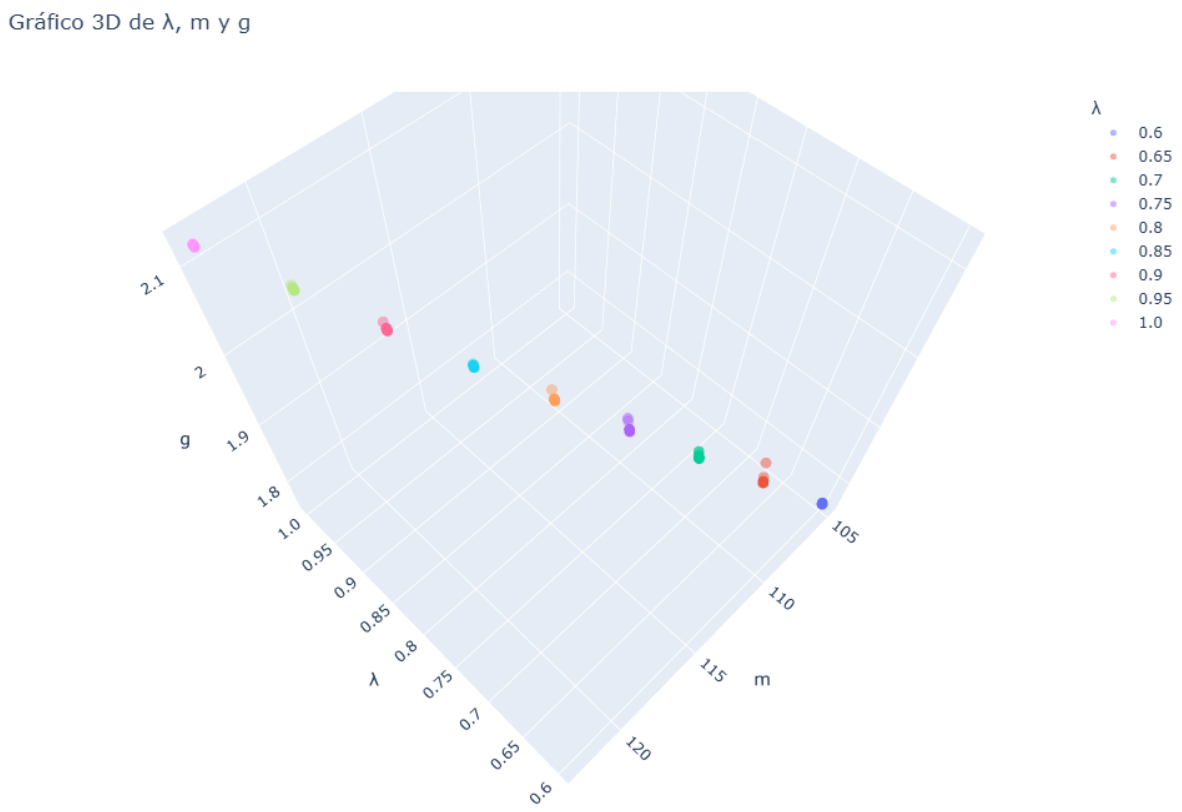

Figura 4: Gráfico 3D desde otra perspectiva donde están todos las combinaciones de parámetros obtenidos desde λ igual a 0.6 hasta 1, en pasos de 0.05 con 100 combinaciones por λ. Donde en el eje vertical está g y en los horizontales λ vs m. Y estamos reportando los puntos que son solución a la curva de Lattice QCD

Podemos observar que los conjuntos de parámetros forman una pequeña recta discreta conforme los valores de λ aumentan. Para visualizar esto más claramente vamos a partir el gráfico en rebanadas de los primeros dos gráficos tridimensionales, por cada valor de λ obtenido. Así podemos visualizar mejor nuestros conjuntos de soluciones. Algunos de los valores tienen a estar muy cerca unos de los otros, y cada combinación se representa con un punto semitransparente; por lo que en los gráficos las regiones más oscuras tienen a tener muchos más valores cercanos.

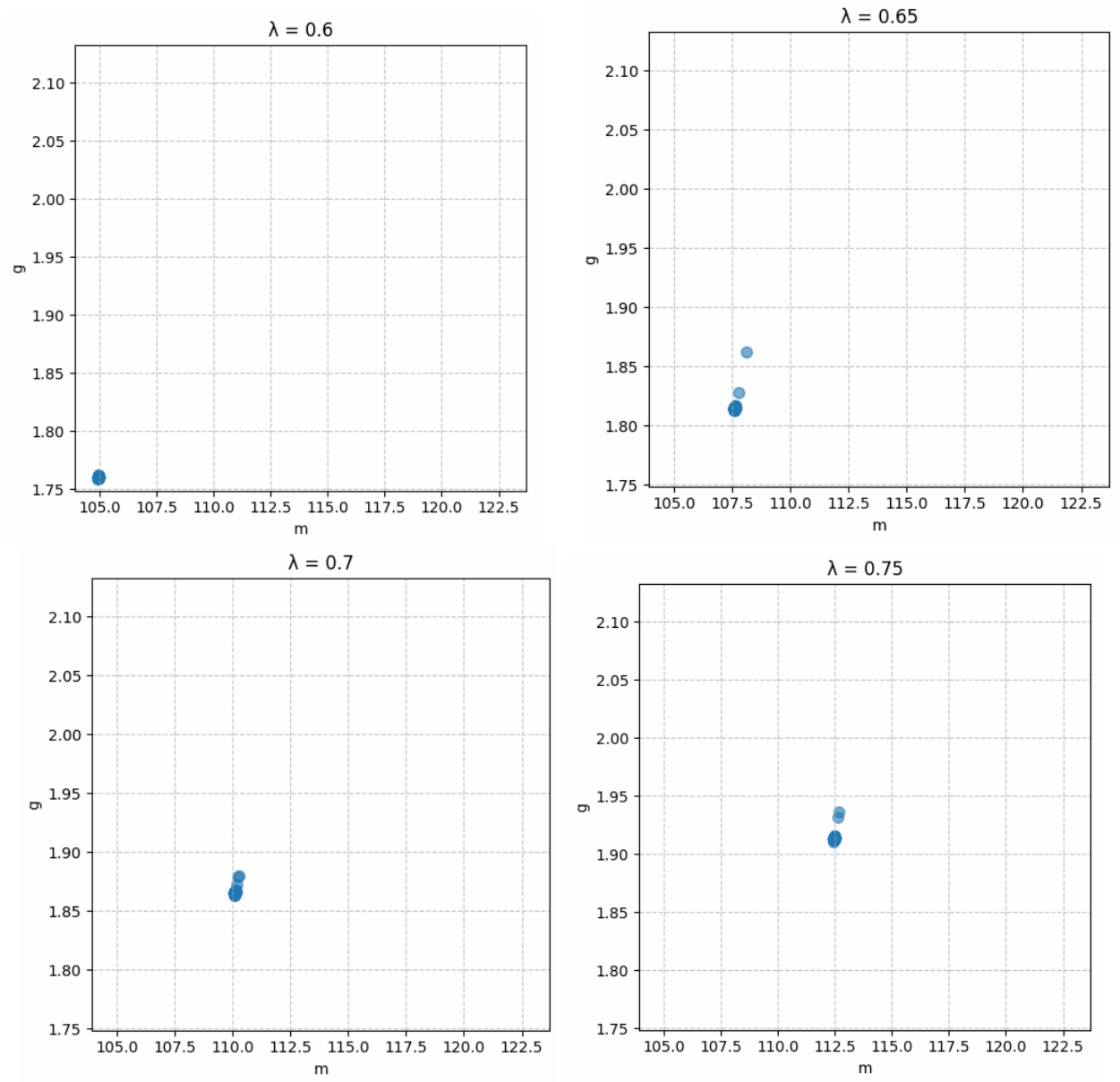

Figura 5: Gráfico de los resultados obtenidos dado un $\lambda = 0.6$ a 0.75, graficando en el eje horizontal m y en el eje vertical g. Donde se observa que al aumentar el λ los conjuntos da parámetros se separan y se desplazan a valores más amplios en ambos ejes.

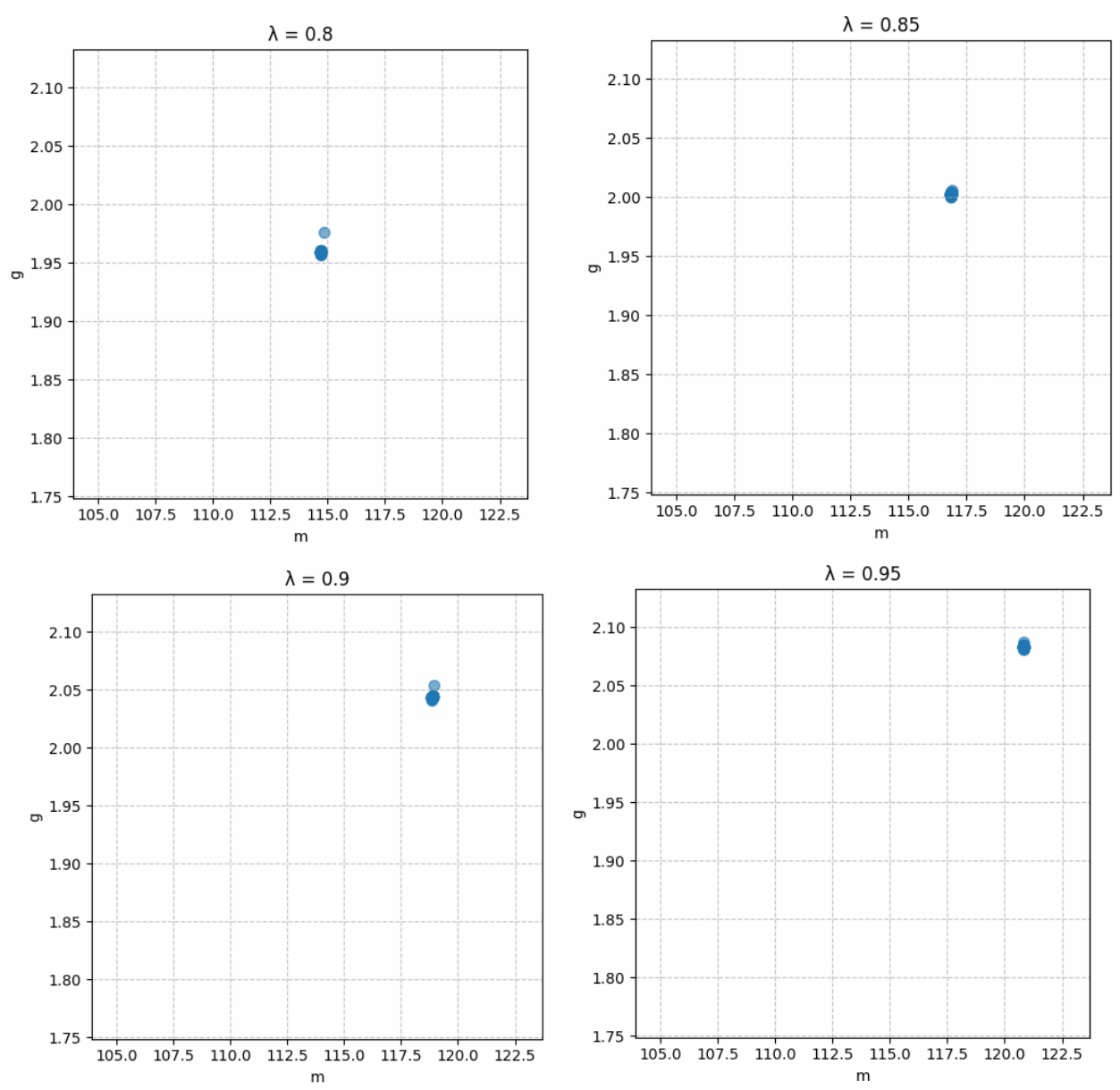

Figura 6: Gráfico de los resultados obtenidos dado un $\lambda = 0.8$ a 0.95, graficando en el eje horizontal m y en el eje vertical g. Donde se observa que al aumentar el λ los conjuntos ahora no se separan tanto, sino que su región en la que se encuentra parece solo desplazarse a valores más amplios en ambos ejes.

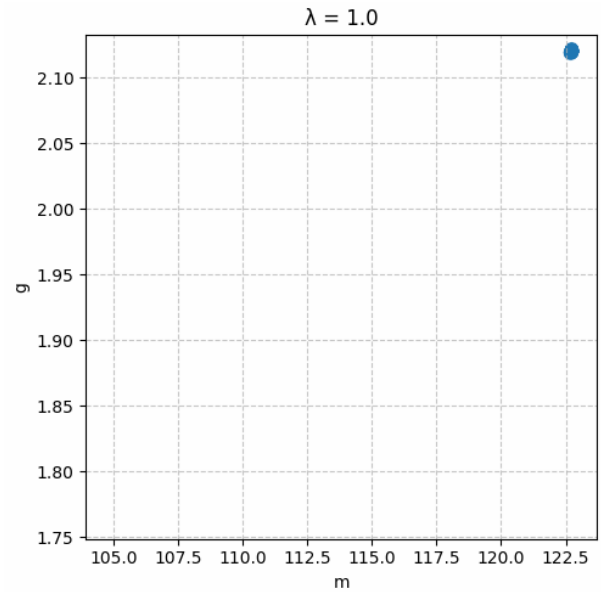

Figura 7: Gráfico de los resultados obtenidos dado un $\lambda = 1$, graficando en el eje horizontal m y en el eje vertical g. Donde dicho parámetro se concentra en una región en específico.

7. Conclusión

Los puntos mostrados en las gráficas cumplen con el objetivo de ser los parámetros óptimos que se aproximen a la curva de LQCD. El número de casos donde la rutina no cumplió con la condición pedida fue de dos combinaciones de novecientas, lo que significa que el programa solo fallo en el 0.002 % de los casos al momento de obtener los parámetros óptimos, estos fallos se encuentran en las combinaciones con $\lambda = 0.8$ y $\lambda = 0.6$, donde estas combinaciones de parámetros no se aproximaron a la curva, sino que tenían un margen de error aproximadamente del 10 %, por lo que fueron omitidas en los gráficos, como se menciono anteriormente. Entre más se aproxime el parámetro λ a 0 aumenta el costo computacional. Es posible observar como los parámetros m y g se concentran en regiones en específico dependiendo del valor de λ. También podrían salir parámetros similares al variar infinitesimalmente en las cifras, debido a la precisión de los optimizadores del código, sin embargo, esto se puede solucionar disminuyendo la tolerancia de error en estas a cambio de un tiempo de ejecución más largo.

Este conjunto de parámetros se observa que tiende a tener el comportamiento de una ligera recta y se podría suponer que se puede aproximar a un continuo, sin embargo, en este proyecto terminal no podríamos afirmar que esto sea correcto o incorrecto debido a las limitaciones tecnológicas y temporales, debido a que se generaron solo novecientas combinaciones de datos, las cuales fueron cien por cada valor de λ desde 0.6 a 1 en pasos de 0.05 y harían falta más combinaciones por cada valor de λ, además, de hacer saltos lo más pequeño posible para tener una mejor estática. Cabe recalcar que, debido al tiempo de ejecución del código,

no hubo tiempo de hacer una estadística más grande, sin embargo, están los retos futuros de optimización y aceleración del código como Numba o CUDA, lo cual el procesamiento en paralelo o concurrente disminuiría enormemente el tiempo de ejecución.

No se usaron redes neuronales debido a que se descubrió que las redes neuronales no disminuirían, en este caso, el tiempo de procesamiento, ya que el cuello de botella estaba en el tiempo de ejecución de las funciones que contienen a nuestro potencial al momento de calcular los valores, además, de que los errores parecían aumentar en el uso de redes neuronales debido a la poca estadística generada anterior. Se podrían usar redes neuronales después de una optimización del código para el análisis de todos los datos generados y, ahora sí, a partir de estos datos usar modelos de *deep learning* para predecir nuevos parámetros.

A pesar de todo esto, los resultados cumplieron con los objetivos propuestos y se espera que este proyecto terminal sea el comienzo (o al menos la motivación) para la búsqueda de muchos nuevos conjuntos de parámetros, tanto en el modelo de Yukawa como en modelos más complejos como el Sigma Lineal de la Cromodinámica Cuántica.

A. Apéndice

El siguiente código de Python calcula los parámetros m y g para cada valor de un cierto *arange* de λ que se puede seleccionar al gusto. Los parámetros m y g, se pueden seleccionar de acuerdo al número de corridas. Se usa un optimizador estocástico llamado *differential evolution* para la obtención de estos parámetros. Ojo, se tarda 4 minutos aproximadamente por una sola obtención de parámetro. En este código al final está la rutina para correr para n λ.

Link al colab: https://colab.research.google.com/drive/1hZ31bEAR-4aECLntafL6E7NCs4wkNegz?usp=sharing

```python
#Aquí importamos todas las paqueterias necesarias para nuestro código

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import roots_legendre
from mpmath import polylog
from scipy.optimize import differential_evolution, root_scalar
from functools import lru_cache


# VARIABLES
# ----------------------------------------------
# Parametros físicos ajustables

m_init = 92.72
g_init = 1.5
lamda = 0.85
mu = 1
T_v = np.linspace(120, 200, 500)
mu_q_values = np.arange(0, 71, 10)

# ----------------------------------------------
# Parametros ajustables de las integrales por Gauss-Legendre

n2_puntos = 100
n3_puntos = 100
nodos2, pesos2 = roots_legendre(n2_puntos)
nodos3, pesos3 = roots_legendre(n3_puntos)


# Celda en la cual estan contenida la expresion de la segunda derivada del potencial evaluada en phi = 0
# ------------------------------------------------------------------------------

@lru_cache(maxsize=1000)                    # Decorador en Python que utiliza la técnica de memorización
def cached_polylog(z):                      # (caching) para optimizar el rendimiento de funciones
    return float(polylog(2, z))

# ------------------------------------------------------------------------------
#Parte del potencial que corresponde a la Autoenergía

def PI(T_v, mu_q, g, lamda):

    T_v = np.asarray(T_v).item()
    exp_1 = -np.exp(mu_q / T_v)
    exp_2 = -np.exp(-mu_q / T_v)
    term_1 = (lamda * T_v**2) / 2
    t_2 = -((g**2 * T_v**2) / np.pi**2) * (cached_polylog(exp_1) + cached_polylog(exp_2))

    return term_1 + t_2

# ------------------------------------------------------------------------------
#Parte del potencial correspondiente al termino número 3 con término integral mediante Gauss-Legendre

def D2_Int_3(T_v, mu_q, nodos2, pesos2, m, g, lamda):

    T_v = np.asarray(T_v).item()
    k_v = (1 + nodos2) / (1 - nodos2)
    dk = 2 / (1 - nodos2)**2
    Pi = PI(T_v, mu_q, g, lamda)
    arg = k_v**2
    raiz = np.sqrt(arg)
    exp_arg_1 = np.exp(-(-mu_q + raiz) / T_v)
    numerador_1 = exp_arg_1 * g**2
    denominador_1 = 1 + exp_arg_1
    exp_arg_2 = np.exp(-(mu_q + raiz) / T_v)
    numerador_2 = exp_arg_2 * g**2
    denominador_2 = 1 + exp_arg_2

    Int_vals = (16 * T_v * lamda) * k_v**2 * (
        -numerador_1 / (denominador_1 * T_v * raiz)
        -numerador_2 / (denominador_2 * T_v * raiz))

    return np.sum(dk * pesos2 * Int_vals)

# ------------------------------------------------------------------------------
#Parte del potencial correspondiente al termino número 4 con término integral mediante Gauss-Legend
```

```
def D2_Int_4(T_v, mu_q, nodos3, pesos3, m, g, lamda):

    T_v = np.asarray(T_v).item()
    k_v = (1 + nodos3) / (1 - nodos3)
    dk = 2 / (1 - nodos3)**2
    Pi = PI(T_v, mu_q, g, lamda)
    arg = -m**2 + k_v**2 + Pi
    raiz = np.sqrt(arg + 0j)
    exp_arg = np.exp(-raiz / T_v)
    numerador = 3 * exp_arg * k_v**2 * lamda
    denominador = 1 - exp_arg

    Int_vals = (8 * T_v * lamda) * (numerador / (denominador * T_v * raiz))

    return np.sum(dk * pesos3 * Int_vals)

# ---------------------------------------------------------------------------
#Función correspondiente que junta y ordena todos los términos anteriores y da el resultado

def Segunda_derivada(T_v, mu_q, lamda, m, g):

    term_1 = 1 / (16 * np.pi**2 * lamda)
    mu_m_ratio = mu**2 / m**2

    if mu_m_ratio <= 0 or mu_m_ratio >= 2:                # Verificar dominio válido
        return np.inf
    term_2 = m**2 * (
        -4 * g**4 - 16 * np.pi**2 * lamda + 9 * lamda**2
        + 3 * lamda**2 * np.log(-mu_m_ratio + 0j)
        - 3 * lamda**2 * np.log(mu_m_ratio / 2)
    )
    term_3 = D2_Int_3(T_v, mu_q, nodos2, pesos2, m, g, lamda)
    term_4 = D2_Int_4(T_v, mu_q, nodos3, pesos3, m, g, lamda)

    return term_1 * (np.real(term_2) - term_3 + term_4)   #Junta todos los términos



#Celda cuya funcion es encontrar la temperaturas crítica

def find_root(mu_q, T_v, lamda, m, g):
    def f(T):
        return np.real(Segunda_derivada(np.array([T]), mu_q, lamda, m, g))    #Regresa la parte real de las egunda derivada
    try:
        sol = root_scalar(f, bracket=[T_v[0], T_v[-1]], method='brentq')      #Busca la raices entre el primer y ultimo valor de T
        return sol.root if sol.converged else np.nan
    except ValueError:         # Manejar casos donde no hay cambio de signo
        return np.nan


# ---------------------------------------------------------------------------
#Encuentra la raiz para distintos valores de mu_q

def raiz(T_v, lamda, m, g,  mu_q_values):
    respuesta = [find_root(mu_q, T_v, lamda, m, g) for mu_q in mu_q_values] # Aquí se usa una "list comprehension" para evaluar la función f
    respuesta = np.array(respuesta)
    if np.isnan(respuesta[0]) or respuesta[0] == 0:   # Verifica si el primer elemento de la respuesta es nan (no es un número)
        return np.full_like(respuesta, np.nan)        # o si es exactamente cero , esto es útil para evitar errores posteriores
    return respuesta                                  # Para indicar que la búsqueda de raíces falló completamente para esa configuración de



# Curva de TF de Latice-QCD y su funcion de perdida, además su discretización.

def Tf_QCD():
  Tc = []
  for i in range(len(mu_q_values)):
    x = 3*mu_q_values[i]
    Tc.append(158 - 0.000096*(x**2) + 8.1129e-11*(x**4))# valores multiplicados por TC_0
  Tc = np.array(Tc)
  return Tc

# ---------------------------------------------------------------------------
# Función de perdida que compara los valores obtenidos con los de la temperatura de Lattice-QCD

def loss(parametros, mu_q_values, lamda):
    m, g = parametros
```

```
    raiz_valores = raiz(T_v, lamda, m, g, mu_q_values)
    if np.any(np.isnan(raiz_valores)):
        return 1e10
    raiz_QCD = Tf_QCD()
    return np.mean((raiz_valores - raiz_QCD)**2)


# Optimización solo para un valor de lamda
# ---------------------------------------------------------------------------
# Definir los límites para los parámetros m y g

bounds = [(50, 150), (0.5, 5)]

# ---------------------------------------------------------------------------
# Ejecutar la optimización

resultados = differential_evolution(
    loss,
    bounds=bounds,
    args=(mu_q_values, lamda),
    tol=1e-3,
    maxiter=100,
    disp=True)

print("Parámetros óptimos encontrados (m, g):", resultados.x)


# Gráfica de comparación entre Lattice QCD y los parametros calculados
# ---------------------------------------------------------------------------
#Valores obtenidos anteriormente

m_opt, g_opt = resultados.x
Tc_opt = raiz(T_v, lamda, m_opt, g_opt, mu_q_values)
Tc_QCD = Tf_QCD()

# ---------------------------------------------------------------------------
# Graficos obtenidos donde ya pasamos a mu_barionicos

plt.plot(3 * mu_q_values, Tc_QCD, 'o-', label='T_c QCD')        # Pasamos a potencial barionico
plt.plot(3 * mu_q_values, Tc_opt, 'x-', label='T_c Calculada')  # Pasamos a potencial barionico
plt.title('Gráfico de comparación entre Lattice QCD y los parametros calculados')
plt.xlabel('μ_barionico (Mev)')
plt.ylabel('T_c (Mev)')
plt.grid(True)
plt.legend()
plt.show()


#Muestra la diferencia numerica entre los resultados obtenidos y los de QCD de las temperaturas críticas

Tc_QCD = Tf_QCD()
plt.plot(3 * mu_q_values, np.abs(Tc_QCD-Tc_opt) , 'ro', label='Valores de las diferencias' )    # Pasamos a barionico
plt.title('Diferencia entre los resultados obtenidos y los de QCD de las temperaturas criticas')
plt.xlabel('μ_barionico (Mev)')
plt.ylabel('Tc_QCD - Tc_opt(Mev)')
plt.legend()
plt.grid(True)
plt.show()


#Funcion de optimización ya ajustada para correr varios lamdas
# ---------------------------------------------------------------------------

def optimizar_para_varios_lambda(lamda_min, lamda_max, paso, num_runs):

    # Generar la lista de valores de mu_q (0, 10, ..., 100)
    mu_q_values = np.arange(0, 71, 10)

    # Lista donde se guardarán todos los resultados
    resultados_opt = []

    # Suponiendo que bounds está definido globalmente como en tu código original
    bounds = [(50, 150), (0.5, 5)]

    # Iterar sobre cada valor de lambda
    for lam in np.arange(lamda_min, lamda_max + 0.01 , paso):
        # Para cada lambda, correr la optimización num_runs veces
        for n in range(num_runs):
            res = differential_evolution(
```

```
            res = differential_evolution(
                loss,                   # Función de pérdida definida en tu código
                bounds=bounds,          # Límites para m y g
                args=(mu_q_values, lam),  # Argumentos: mu_q_values y el lambda actual
                maxiter=100,            # Máximo de iteraciones
                disp=False              # Sin mostrar el progreso de cada optimización
            )

            m_opt, g_opt = res.x
            # Agregar la tupla (lam, m_opt, g_opt) a la lista de resultados
            resultados_opt.append((lam, m_opt, g_opt))

        # Imprimir un mensaje al completar las 1000 corridas para este lambda
        print(f"Completadas {num_runs} corridas para λ = {lam:.2f}")

    return resultados_opt

resultados = optimizar_para_varios_lambda(lamda_min=1, lamda_max=1.3, paso=0.1, num_runs=50)


print(resultados)
```

El código de Python siguiente es para comprobar los datos teóricos de Lattice QCD vs los obtenidos mediante el código anterior. Aquí se pueden sustituir cualquier combinación de parámetros iniciales y mostrará la gráfica visual de Lattice QCD y la construida mediante el código, además de mostrar una gráfica con los valores de la diferencia entre estas dos gráficas discretizadas de las temperaturas críticas.

Link al colab: https://colab.research.google.com/drive/1sxcd_N9q-lUlIPHAckYlWVmuxnET9lMJ?usp=sharing

width=!,height=!,pages=-

Referencias

[1] Alejandro Ayala, Saul Hernandez-Ortiz, and Luis A. Hernandez. QCD phase diagram from chiral symmetry restoration: analytic approach at high and low temperature using the Linear Sigma Model with Quarks. *Rev. Mex. Fis.*, 64(3):302–313, 2018.

[2] Alejandro Ayala, Bilgai Almeida Zamora, J. J. Cobos-Martínez, S. Hernández-Ortiz, L. A. Hernández, Alfredo Raya, and María Elena Tejeda-Yeomans. Collision energy dependence of the critical end point from baryon number fluctuations in the Linear Sigma Model with quarks. *Eur. Phys. J. A*, 58(5):87, 2022.

[3] Vargas Fernández Ana G. Diagrama de fase efectivo de la intereacción fuerte en ppresencia de campos magnéticos externos. Master's thesis, Universidad Autónoma Unidad Iztapalapa, 5 junio, 2025.

[4] Michael E. Peskin and Daniel V. Schroeder. *An Introduction to Quantum Field Theory.* Westview, Boulder, CO, 1995. Includes exercises.

[5] Joseph I. Kapusta and Charles Gale. *Finite-Temperature Field Theory: Principles and Applications.* Cambridge University Press, 2 edition, 2006.

[6] Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri. *Numerical Mathematics.* Springer-Verlag, Berlin Heidelberg, second edition, 2006.

[7] Courville A Goodfellow Ian, Bengio Y. *Deep Learning.* Massachusetts Institute of Technology, 2016.

[8] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.

[9] L. Dolan and R. Jackiw. Symmetry Behavior at Finite Temperature. *Phys. Rev. D*, 9:3320–3341, 1974.

[10] Mishra A. *Deep learning with tensorflow and keras from fundamentals to advanced architectures.* Anshuman Mishra, 2025.

[11] Jana N. Guenther. Overview of the QCD phase diagram: Recent progress from the lattice. *Eur. Phys. J. A*, 57(4):136, 2021.